

# 작업 계획

## 실험이 성립하기까지

인프라는 끝났다. 실험은 원본에서 끊긴 피드백 루프 하나 때문에 아직 시작되지 못했고, 그 위에 얹힌 개입 판정 공식은 위험을 재지 않아 논문의 핵심 지표가 지금은 무의미하다.

92%

SLA 위반 중  
배분 탕

0.49

개입 판정의  
SLA 예측력 (AUC)

3

코드 전에  
정할 결정

# 지금 어디까지 왔나

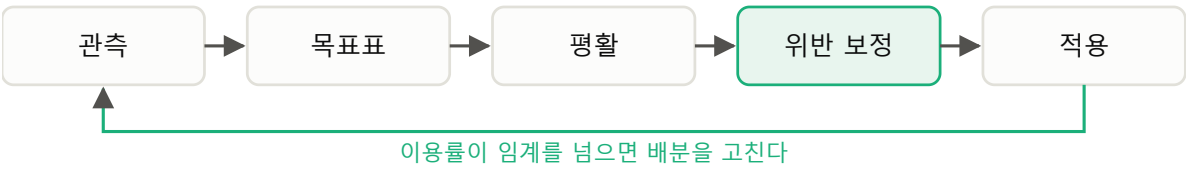
|                                  |             |                       |
|----------------------------------|-------------|-----------------------|
| 배선 · 서버 5개 · 드라이버 2종 · 채점기 · 재현성 | <div></div> | 완료                    |
| 실험 장치 · 비교군 · 집계 · 배치            | <div></div> | arms/ · run_matrix 없음 |
| 결정 · 위반 보정 · 개입 공식 · 조달 시점       | <div></div> | 회의 필요                 |
| 결과 · 논문 표에 들어갈 수치                | <div></div> | 측정 불가 상태              |

"구축은 끝났고 나머지는 조정"은 절반만 맞다. 조정 중 하나(개입 판정 공식)가 논문의 축을 흔들 수 있다.

# 왜 지금 결과가 이런가

## 원본에서 끊긴 것

원본 ml\_orchestrator\_demo.py:429~461 (한 함수)



현재 ② rule.py + ① env.py (서버 두 개로 분리)



평활이 ①로 가면서 보정을 걸 자리가 사라졌다 → ②는 관측을 읽지 않는다 → 출력 4가지

## 그 위에 쌓인 것

1층 · 기원

위반 보정 누락

배분이 관측에 반응 못 함 → SLA 위반의 92%가 배분 탕

2층 · 증폭

개입 스텝 귀속 오류

폴백 상수의 실패가 정책 점수로 → r 0.50 → 0.20

3층 · 지표 무효

개입 판정 공식

선행 위험 신호가 없음 → AUC 0.49~0.68 → 논문 지표 무의미

개입 → 폴백 실패 → r↓ → 더 개입

## 근거 — 전부 실서버 실측

| 관측                    | 값             | 뜻                                      |
|-----------------------|---------------|----------------------------------------|
| SLA 위반 중 압력>1.0(구조적)  | 1 / 12        | 나머지 11건은 배분이 관측에 반응하지 않아서              |
| 개입 스텝의 적용 배분 = 폴백 상수  | 18/18 · 19/19 | 성적표의 60%가 정책이 하지 않은 일의 점수              |
| rule_based 신뢰도 r      | 0.500 → 0.200 | 120스텝 동안 붕괴 — 정책이 나빠서가 아니라 개입했기 때문     |
| combined 의 SLA 예측력    | 0.683 / 0.487 | normal / emergency. 0.5 = 무작위          |
| intrinsic 단독          | 0.487 / 0.463 | 분기 명확성을 쟁 켜 위험이 아님                     |
| worst u/θ (관측에 있는 값)  | 0.857 / 0.608 | 가장 좋은 신호가 공식 밖에 있다                     |
| lstm_forecast 선택 → 개입 | 47 / 47       | 표본이 영원히 0. 정책 선택 실증 불가                 |
| 조달 제약(1.0) 전 → 후      | 5회 → 1회       | 스텝당 \$0.109 → \$0.082. 두 드라이버 비교가 공정해짐 |

# 먼저 정해야 할 것 — 코드보다 먼저

## D1 · 위반 보정

원본 :446~457 . 임계를 넘은 슬라이스에 더 주고 여유 있는 데서 뻔다. kim이 "Day 0에 3인이 정할 사항"으로 남겨둔 것.

**권고** 환경변수 스위치. 기존 기준은 컴(원본 동작). 끈 값은 상황인지 순도 논증용.

## D2 · 개입 판정 공식

$\sqrt{(\text{intrinsic} \times \text{empirical})} < 0.45$  . 원본에 없던 신규 설계. 두항 모두 선행 위험 신호가 아니다.

**권고** 귀속 오류(B-2)를 고친 뒤 **다시 켜 값**으로 정한다.  $\tau=0.45$  는 유지하고 민감도를 보고.

## D3 · 조달 시점

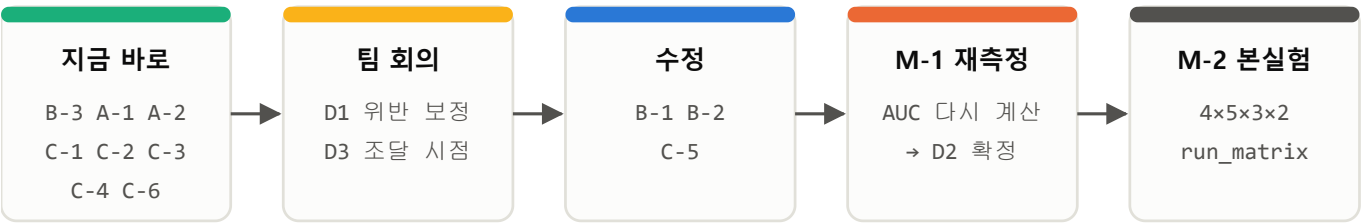
1.0을 넘은 뒤 사면 용량은 다음 스텝에 반영 → 그 스텝 위반 확정. 반응형이다.

**권고** 고정 루프는 1.0 유지(기존 기준), 오케스트레이터만 추세 선제 허용 → **그 차이가 기여**. 단 D1 이후.

# 누가 무엇을

|                                                                                                                                                                                                                               |                                                                                                                                   |                                                                                                                                                                                                                                                                                                                            |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div>B · kim · ②③⑤</div> <div><div>B-1 policy/rule.py 위반 보정 이식 ← D1</div><div>B-2 feedback/server.py:143 개입 스텝은 r 갱신 제외</div><div>B-3 feedback/reliability.py:60 lstm 봉인 해제</div><div>B-4 rule.py "안전 하한" 독스트링 정정</div></div> | <div>A · choi · ①④</div> <div><div>A-1 audit/book.py:234 agent_policy 필드</div><div>A-2 audit/book.py confidence 타입 검사</div></div> | <div>C · lee · 에이전트</div> <div><div>C-1 intent 를 장부에 기록</div><div>C-2 eval/score.py 구조적·오류 스텝 분리</div><div>C-3 referee.py 개입 무시 규칙</div><div>C-4 arms/ 3종 · escalate 필드화</div><div>C-5 tools/run_matrix.py</div><div>C-6 vendors.json 추적 해제</div><div>C-7 개입 공식 ← D2 · M-1</div><div>C-8 선제 조달 프롬프트 ← D3 · B-1</div></div> |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 순서



D2(개입 판정 공식)는 M-1 결과를 보고 정한다 — 귀속 오류가 겹친 뒤의 값이어야 한다

지금 바로 줄은 반나절이고 서로 독립이다. 회의 전에 끝내두면 D1이 나는 순간 B-1 → M-1로 간다. 결정 전에 본실험 60회를 돌리면 전부 다시 돌리게 된다.

## 부록 · 작업 계획 원문

claude/team/workplan.md — AI 가 그대로 실행할 수 있게 쓴 것

작성 2026-09-24 · C(에이전트) · 대상 A·B·C 전원과 이 저장소에서 일하는 AI

### 0. 이 문서를 읽는 AI 에게

**목적.** 논문 주장 "개입은 줄었는데 SLA 위반은 늘지 않았다" 를 측정 가능하게 만든다. 지금은 측정이 불가능하다. 이 문서는 그 이유와, 가능해질 때까지의 작업을 담당·파일·줄·검증 명령까지 적는다.

**전제.** - 실행은 .venv310\Scripts\python.exe 로만 한다. PATH 의 3.14 에는 fastmcp · tensorflow 가 없다. - 담당 표기: **A**=choi(①④·측정) · **B**=kim(②③⑤) · **C**=lee(에이전트· eval · tools ) - **자기 담당이 아닌 파일은 고치지 않는다.** 여기 적힌 변경은 해당 담당이 한다. - 근거 수치는 runs/ 의 실행 산출물에서 나왔다. 재현 명령은 각 항목에 있다. - 이 문서보다 자세한 원인 분석은 issue-B-policy-feedback.md (8건) 에 있다. 우선순위·담당 표는 todo-after-orchestrator.md (14건) 에 있다. 이 문서는 그 둘을 **실행 순서**로 다시 엮은 것이다. 셋이 어긋나면 이 문서가 최신이다.

**하지 말 것.** - 결정 D1·D2·D3 이 나기 전에 본실험(60회)을 돌리지 않는다. 전부 다시 돌리게 된다. - 결과가 좋아질 때까지 상수( $\tau$ ·평활·임계)를 돌리지 않는다. 민감도로 보고한다. - --intent 에 상황 라벨("지금 emergency 다")을 주 실험에 넣지 않는다. 오라클 비교군에만.

### 1. 진단 — 세 층

- 1층 분해 과정에서 피드백 루프가 끊겼다  
원본 m1\_orchestrator\_demo.py:446~457 (위반 보정) 이  
a(평활·클립) 과 b(목표표) 사이에 빠짐  
→ a.propose() 가 observation 을 안 읽음 → 출력 4가지  
→ SLA 위반의 92% 가 배분 탕 (압력>1.0 은 8%)

← 기원
- 2층 실패가 엉뚱한 주체에게 귀속된다  
개입 스텝은 폴백 상수 {0.4,0.4,0.2} 를 적용하는데  
c server.py:143 이 그 결과를 rule\_based 의 r 에 기록  
→ 개입 → 폴백 실패 → r↓ → 더 개입 (자기강화)  
→ r 0.500 → 0.200 (120스텝)

← 증폭기
- 3층 개입 판정 공식이 위험을 재지 않는다  
combined =  $\sqrt{\text{intrinsic} \times \text{empirical}}$   
intrinsic(rule) =  $0.5 + 0.3 \cdot \min_k |u_k - \theta_k| / \theta_k$  → 분기 명확성, 위험 아님  
empirical = EMA(과거 sla\_met) → 후행 지표  
→ 실패를 예측 못 함 (AUC 0.487~0.683)  
→ 관측에 있는 worst u/ $\theta$  는 AUC 0.857 인데 공식 밖

← 논문 지표 무효

#### 근거 (전부 실서버 · 재현 가능)

| 관측                   | 값             | 실행                                       |
|----------------------|---------------|------------------------------------------|
| SLA 위반 중 압력>1.0(구조적) | 1/12          | prompttest-mixed-s0 12스텝 orchestrator    |
| 개입 스텝의 적용 배분 = 폴백    | 18/18 · 19/19 | measure-normal-s0 · measure-emergency-s0 |
| rule_based r 붕괴      | 0.500 → 0.200 | 120스텝 mixed (덮어써짐, 리포트에 기록)              |

| 관측                       | 값                         | 실행                      |
|--------------------------|---------------------------|-------------------------|
| combined 의 SLA 예측력 (AUC) | 0.683 / 0.487             | normal / emergency 30스텝 |
| intrinsic 단독             | 0.487 / 0.463             | 같음                      |
| worst u/θ (관측값)          | <b>0.857 / 0.608</b>      | 같음                      |
| lstm_forecast 선택 → 전부 개입 | 47/47, n 은 0              | 120스텝 mixed             |
| 조달 제약(1.0) 전후            | 5→1회 · \$0.109→\$0.082/스텝 | 12스텝 orchestrator ×2    |

재현:

```
.venv310\Scripts\python.exe run.py --backend mcp --decider rule --scenario emergency --steps 30 --arm
measure --fresh --memory-mode cold
.venv310\Scripts\python.exe -m eval.score measure-emergency-s0
```

(AUC 는 decisions.json 의 confidence.combined 와 outcome.sla\_met 로 계산한다. 스크립트는 §6 참조.)

## 2. 먼저 정해야 할 것 — 코드 전

### D1. 위반 보정을 되살릴 것인가

|           |                                                                                    |
|-----------|------------------------------------------------------------------------------------|
| 무엇        | 원본 :446~457 — 임계 넘은 슬라이스에 +min(0.1, 초과분×0.2), 가장 여유 있는 슬라이스에서 뺌                    |
| 왜 빠졌<br>나 | policy/rule.py 모듈 독스트링 — 평활이 ①로 가서 걸 자리가 사라짐. "Day 0 에 3인이 정할 사항"                  |
| 커면        | 배분이 관측에 반응. SLA 위반 63~92% 가 내려갈 것                                                  |
| 고면        | 상황 라벨 오판이 보정으로 가려지지 않아 상황인지 신호가 깨끗함                                                |
| 권고        | 환경변수 스위치 SLICE_RULE_CORRECTION=on/off . 기존 기준은 on (원본 동작). off 는 상황인지 순도 논증용<br>대조 |
| 여는 작<br>업 | B-1                                                                                |

### D2. 개입 판정 공식을 어떻게 할 것인가

|     |                                                                                                       |
|-----|-------------------------------------------------------------------------------------------------------|
| 무엇  | combined = √(intrinsic × empirical) < τ(0.45) — 설계서 flow/data-chain.md:107 . 원본에는 없던 신<br>규 설계        |
| 문제  | 두 항 모두 선행 위험 신호가 아님 (§1 3층). AUC ~0.5                                                                 |
| 주의  | 2층(귀속 오류)이 empirical 을 오염시키고 있음. B-2 이후 재측정(M-1)한 값으로 판단한다                                            |
| 선택지 | (a) 유지하고 민감도만 보고 (b) worst u/θ 같은 선행 신호를 게이트로 추가 (c) 공식 교체                                            |
| 권고  | <b>M-1 결과로 정한다.</b> 재측정 AUC 가 0.7 이상이면 (a). 0.6 미만이면 (b). 어느 쪽이든 τ=0.45 는 유지하고 0.3~0.6 민감<br>도를 같이 보고 |

### D3. 조달 시점

무엇 `PROCURE_PRESSURE = 1.0` ( `spec/observe.md:27` ). 압력이 1.0 을 넘은 뒤 사면 용량은 다음 스텝에 반영 → 그 스텝 위반 확정

선택지 (a) 양쪽 다 1.0 반응형 (b) 고정 루프 1.0 유지 · 오케스트레이터는 이력 추세로 선제 허용

권고 (b). 고정 루프는 기존 기준 그대로, 오케스트레이터에만 재량 → 그 차이가 기여. 단 **B-1 이후**에 켜다 (지금은 위반의 92% 가 조달과 무관해 효과가 안 보임)

### 3. 작업 목록

형식: ID · 담당 · 파일:줄 → 변경 → 검증 → 의존

#### B (kim · ②③⑤)

**B-1 · `srm_mcp/policy/rule.py` `propose()`** — 의존: D1 → 원본 :446~457 을 옮긴다. `observation["utilization"]` 과 `THRESHOLDS` 로 임계 초과 슬라이스에 `increase = min(0.1, (u-θ)×0.2)` 를 더하고 `utilization` 이 가장 낮은 슬라이스에서 뺀다. `SLICE_RULE_CORRECTION` 환경변수(기본 on )로 켜고 끈다. 평활·클립은 넣지 않는다(①의 몫). → 검증: emergency 30스텝에서 `propose_allocation` 반환 `allocation` 이 스텝마다 달라진다 (지금은 4가지 상수). SLA 위반 이 21/30 아래로 내려간다. → 주의: 원본은 *평활된* 배분에 보정을 걸었다. ②는 현재 배분을 모르므로 목표표에 직접 건다. 이 차이를 독스트링에 적는다.

**B-2 · `srm_mcp/feedback/server.py:143~163`** — 의존: A-1 (권장, 필수 아님) → `record.get("escalated")` 가 참이면 `reliability.update()` 와 `push_error()` 를 건너뛴다. `outcome` 은 그대로 돌려준다(④ 채점은 남아야 함). 폴백 성능 을 따로 보려면 `table["fallback"]` 항목을 두고 거기 기록한다(선택). → 검증: `measure-emergency-s0` 재실행 후 `reliability.json` 의 `rule_based.n` 이 개입 횟수만큼 적다. 개입 19회면 `n=11`. → 근거: `issue-B-policy-feedback.md` 5번.

**B-3 · `srm_mcp/feedback/reliability.py:53~61`** — 의존: 없음 → `errors` 가 비면 `None` 대신 `reliability` 사전값에 서 유도한 오차를 준다. `reliability.py` 의 `initial_entry()` 가 `r=0.5` 를 주므로, 정책별 사전값이 필요하다면 `POLICY_PRIOR = {"rule_based":0.9, "lstm_forecast":0.8, "dqn":0.4}` 를 `const.py` 에 두고 `recent_error = 1 - prior` 로 유도한다 (`lstm` → 0.2 → ② `conf exp(-3×0.2)=0.549 > τ`). → 검증: normal 30스텝 규칙 판단자에서 스텝 10 이후 `lstm_forecast` 의 `confidence` 가 0.2231 이 아니다. 실행 끝에 `lstm_forecast.n > 0` . → 근거: 같은 문서 1번. `reliability.py:57` 의 유예 가정에 구멍이 있다(`errors` 는 정책별).

**B-4 · `srm_mcp/policy/rule.py` `confidence()` 독스트링** — 의존: 없음 → "*하한 0.50 이라 항상  $\tau$  위*" 를 지운다. `combined` 는 `empirical` 이 곱해지므로 보장이 아니다. → 근거: 같은 문서 2번.

#### A (choi · ①④)

**A-1 · `srm_mcp/audit/book.py:172~250`** — 의존: 없음 → `record_escalation` 시그니처에 `chosen_policy`: `Optional[str] = None` 을 받고, 폴백 결정 레코드에 `"agent_policy": chosen_policy` 를 추가한다. `"chosen_policy"`:

FALLBACK\_POLICY 는 그대로 둔다(⑤가 실행된 정책을 알아야 함). situation 을 보존한 :185 의 논리와 같다. → 검증: 오케스트레이터 12스텝 후 decisions.json 의 개입 레코드에 agent\_policy 가 있고, 루프 요약의 정책 사용 과 장부의 agent\_policy 집계가 일치한다. → 근거: 같은 문서 4번. 에이전트(C-1)가 값을 넘긴다.

A-2 · srm\_mcp/audit/book.py bad\_confidence() — 의존: 없음 → 네 키의 값이 float 로 변환되는지 확인한다. 아니면 malformed\_confidence 로 거부. → 근거: todo-after-orchestrator.md 8번 (haiku 가 문자열을 넣은 실측).

## C (lee · 에이전트 · eval · tools)

C-1 · run.py · agent/loop.py · agent/orchestrator/host.py — 의존: 없음 → --intent 값을 ④ record\_decision / record\_escalation 의 config 에 넣어 장부에 남긴다. record\_escalation 에 chosen\_policy=decision.policy 를 넘긴다(A-1 이 받으면 기록됨, 안 받으면 무시됨 — Tools.\_call 이 None 만 거르므로 호환). → 검증: decisions.json["config"] 에 intent 키.

C-2 · eval/score.py — 의존: 없음 → (a) demand\_pressure > 1.0 인 스텝을 structural 로 따로 센다. 제외가 아니라 분리. (b) runs/<id>/orchestrator/referee.jsonl 이 있으면 error 등급 위반 스텝을 perception\_accuracy · escalation\_precision 에서 빼고 제외 수를 보고한다. → 검증: measure-emergency-s0 채점 결과에 structural: 8 이 나온다. → 근거: 같은 문서 8번 · todo 6번.

C-3 · agent/orchestrator/referee.py:64~147 — 의존: 없음 → compute\_confidence 가 escalate:true 를 냈는데 같은 스텝에 record\_decision 이 있으면 ignored\_escalation (error) 를 낸다. 호출 기록의 반환값을 봐야 하므로 calls.jsonl 의 result 필드를 쓴다. → 검증: 검증 스크립트로 그런 스텝을 만들어 위반이 잡힌다. → 근거: todo 7번.

C-4 · agent/schema.py:97~105 · agent/arms/ 신규 — 의존: 없음 → Decision.escalate 를 파생 속성에서 필드로 바꾼다(기본값은 지금 공식). arms/ 에 baseline.py (사람이 --emergency 로 상황 지정, truth.jsonl 읽는 유일한 경로, 물리 분리) · arm1.py (상황=에이전트, 정책 고정, escalate 항상 True) · arm2.py (정책도 에이전트, escalate 항상 True) 를 둔다. run.py --arm 이 이걸 고른다. → 검증: 세 arm 이 같은 시드에서 같은 스텝 수를 소화하고, baseline 만 Guard(enabled=False) . → 근거: origin/decomposition.md §6 비교군 표 · flow/forbidden.md:17 .

C-5 · tools/run\_matrix.py 신규 — 의존: C-4 → 비교군 4 × 시나리오 5 × 시드 3 을 순서대로 돌리고 runs/matrix-<날짜>/summary.json 과 각 report.html 을 모은다. 비용 상한( --budget-usd )과 재시작 지점( --resume )을 갖는다. 드라이버는 fixed · orchestrator 를 인자로. → 검증: --dry-run 이 60개 명령을 출력한다.

C-6 · git — 의존: 없음 → git rm --cached data/vendors.json . .gitignore 에 이미 있다. 부트스트랩이 원본에서 재생성. → 검증: git status 가 실행 후에도 깨끗하다.

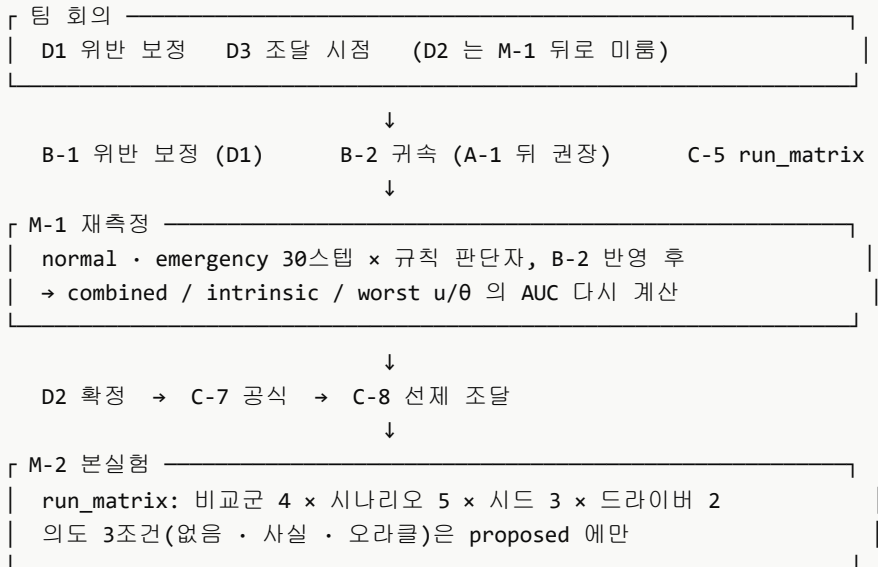
C-7 · agent/schema.py · agent/orchestrator/gateway.py:294~301 — 의존: D2 · M-1 → D2 결과대로. (b) 라면 두 곳에 같은 게이트를 넣는다 — 한쪽만 바꾸면 드라이버 비교가 깨진다. 단위 검증 스크립트가 두 구현이 같은 입력에 같은 출력을 내는지 확인한다. → 검증: check\_orchestrator.py 의 compute\_confidence 공식 항목이 새 공식으로 통과.

C-8 · agent/orchestrator/prompt.md — 의존: D3 · B-1 → (b) 라면 조달 절에 "get\_history 의 추세가 상승이고 다음 스텝에 1.0 을 넘을 것으로 보이면 선제 조달할 수 있다" 를 넣는다. 고정 루프는 건드리지 않는다. → 검증: 압력 상승 구간에서 조달 스텝이 1.0 도달 전에 나타난다.

## 4. 순서

|                   |           |                   |          |         |  |
|-------------------|-----------|-------------------|----------|---------|--|
| 지금 바로 (결정 무관, 병렬) |           |                   |          |         |  |
| B-3 lstm 부트스트랩    |           | B-4 독스트링          |          |         |  |
| A-1 agent_policy  |           | A-2 confidence 타입 |          |         |  |
| C-1 intent 기록     | C-2 채점 분리 | C-3 심판 규칙         | C-4 arms | C-6 git |  |

↓



지금 바로 줄은 반나절이고 서로 독립이다. 회의 전에 끝내두면 D1 이 나는 순간 B-1 → M-1 로 간다.

## 5. 완료 판정

각 단계가 끝났다고 말하려면 아래가 참이어야 한다.

단계 판정

|     |                                                                                                              |
|-----|--------------------------------------------------------------------------------------------------------------|
| B-1 | <code>emergency</code> 30스텝 SLA 위반 < 21/30, <code>propose_allocation</code> 출력이 스텝마다 다름                      |
| B-2 | 개입 19회 실행 후 <code>rule_based.n == 30 - 19</code>                                                             |
| B-3 | 30스텝 후 <code>lstm_forecast.n &gt; 0</code>                                                                   |
| A-1 | 개입 레코드에 <code>agent_policy</code> 존재, 루프 집계와 일치                                                              |
| C-2 | 채점 결과에 <code>structural</code> 과 <code>excluded_by_referee</code> 키                                          |
| C-4 | <code>arms/baseline.py</code> 만 <code>truth.jsonl</code> 을 연다 ( <code>grep -l truth agent/</code> 가 그 파일 하나) |
| M-1 | <code>combined</code> AUC 가 두 시나리오에서 기록됨. 값이 무엇이든                                                            |
| D2  | M-1 값과 선택 근거가 이 문서 §2 에 추가됨                                                                                  |
| M-2 | <code>runs/matrix-*/summary.json</code> 에 60행 이상, 각 행에 개입률·SLA·비용·perception-escalation_precision            |

## 6. 참조

- 원인 상세: `claude/team/issue-B-policy-feedback.md` (8건, 실측 · 선택지)
- 우선순위 표: `claude/team/todo-after-orchestrator.md` (14건)
- 비교군 정의: `claude/origin/decomposition.md` §6
- 정답 차단: `claude/flow/forbidden.md`
- 실행법 · 환경: `agent/README.md` · `web/serve.py` (실행 UI)

- AUC 계산: `runs/<id>/decisions.json` 의 `kind=="decision"` 레코드에서 (`confidence.combined`, `outcome.sla_met`) 를 모아 Mann-Whitney U / (`n_pos`·`n_neg`). `worst u/θ` 는 `max_k observation.utilization[k] / thresholds[k]` .